Accessibility Testing with Playwright

© 2024 YASH Technologies | www.yash.com | Confidential

Agenda

- What is Accessibility Testing?
- Installation of axe-playwright package for Accessibility
- Scanning an entire webpage for Accessibility
- Scanning specific part of the page for Accessibility
- Scanning for WCAG violations
- Excluding individual elements from a scan
- Disabling individual scan rules
- Exporting scan results as a test attachment
- Generate HTML reports

What is Accessibility Testing?

- Accessibility testing ensures that web content and applications are usable by people with disabilities.
- Automated accessibility testing tools can identify and detect some common accessibility issues with websites or applications, such as:
  - Text that would be hard to read for users with vision impairments due to poor color contrast with the background behind it
  - UI controls and form elements without labels that a screen reader could identify
  - Interactive elements with duplicate IDs which can confuse assistive technologies
- These tools can help developers find and fix problems more efficiently and can be especially useful for maintaining accessibility standards during development.
- Many accessibility problems can only be discovered through manual testing.
- It is recommended to use a combination of automated testing, manual accessibility assessments, and inclusive user testing.

Installation of axe-playwright package for Accessibility

- @axe-core/playwright package adds support for running the axe accessibility testing engine as part of Playwright tests.
- This module uses a chainable API to assist in injecting, configuring, and analyzing axe with Playwright.
- Run the following npm command in the terminal to install the required libraries.

➔ npm i @axe-core/playwright --save-dev

```
D:\PlaywrightWorkspace>npm i @axe-core/playwright --save-dev
```

- As such, axe-core is required to pass an instance of Playwright.

```
const builder = new AxeBuilder({ page });
```

Scanning an entire webpage for Accessibility

- This code demonstrates how to test an entire page for automatically detectable accessibility violations.
- Axe Analysis:
  - *new AxeBuilder({ page }):* Initializes Axe to scan the current page.
  - *analyze():* Performs the accessibility audit.
- **Assertion:** *expect(results.violations).toEqual([])* ensures that no accessibility violations are found.
- Playwright will print a detailed report of any violations found, including the element, the rule violated, and the impact on users at console.
- You can generate a visually appealing HTML report using:
  npx playwright test --reporter=html

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test.describe('Playwright Homepage', () => {

  test('Verify there is no accessibility issues', async ({ page }) => {

    await page.goto('https://playwright.dev');

    const accessibilityScanResults = await new AxeBuilder({ page }).analyze();

    expect(accessibilityScanResults.violations).toEqual([]);

  });

});
```

Scanning specific part of the page for Accessibility

- One can use AxeBuilder.include() to constrain an accessibility scan to only run against one specific part of a page.
- Specify this option by using a Builder pattern with the AxeBuilder class.
- To scan parts of a page that are revealed based on UI interactions, use Locators to interact with the page before invoking analyze() function.
- The results of the accessibility scan are stored in the JSON constant 'accessibilityScanResults'.
- If this array is empty, it indicates that there are no accessibility issues on the page or component, and the test will pass.
- Conversely, if there is at least one accessibility issue, the array will not be empty, and the test will fail.

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test.describe('Playwright Homepage', () => {

  test('Verify there is no accessibility issues', async ({ page }) => {

    await page.goto('https://playwright.dev');

    await page.locator('#id').waitFor();

    const accessibilityScanResults = await new AxeBuilder({ page })
    .include('#id')
    .analyze();

    expect(accessibilityScanResults.violations).toEqual([]);

  });

});
```

Scanning for WCAG violations

- Web Content Accessibility Guidelines (WCAG) are well-known recommendation set for web accessibility.
- WCAG and other rules are best practice axe checks against a wide variety of accessibility rules.
- One can constrain an accessibility scan to run only those rules which are "tagged" as WCAG success criteria by using AxeBuilder.withTags().
- The Tags are : wcag2a, wcag2aa, wcag21a, wcag21aa

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test.describe('Playwright Homepage', () => {

  test('Verify there is no accessibility issues', async ({ page }) => {

    await page.goto('https://playwright.dev');

    const accessibilityScanResults = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
    .analyze();

    expect(accessibilityScanResults.violations).toEqual([]);

  });

});
```

Note: Automated testing cannot detect all types of WCAG violations and there should be manual verification as well.

Excluding individual elements from a scan

- We have the ability to exclude certain components from accessibility testing if they are known issues or deemed unnecessary to test.
- One can use AxeBuilder.exclude() to exclude them from being scanned until you're able to fix the issues.
- This will exclude the specified elements and all of their descendants, so avoid using it with components that contain many children.

```
1  import { test, expect } from '@playwright/test';
2  import AxeBuilder from '@axe-core/playwright'; // 1
3
4  test('test', { tag: '@access7'}, async ({ page },testInfo) => {
5    await page.goto("https://app.box.com");
6    // await page.locator("id=login-email").fill('qacult.demo@gmail.com');
7    await page.getByLabel('Email Address', { exact: true }).fill('qacult.demo@gmail.com');
8
9    await page.locator("id=login-submit").click();
10
11   await page.locator("id=password-login").fill('testing123');
12   await page.getByRole('button', { name: 'Log In' }).click();
13
14   await page.locator("button[data-testid='new-item-menu-button']").waitFor();
15
16   const accessibilityScanResults = await new AxeBuilder({ page })
17       .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
18       .exclude("button[data-testid='new-item-menu-button']")
19       .analyze();  // 4
20
21       await testInfo.attach('accessibility-scan-results', {
22         body: JSON.stringify(accessibilityScanResults, null, 2),
23         contentType: 'application/json'
24       });
25
26     expect(accessibilityScanResults.violations).toEqual([]); // 5
27   // await page.close();
28
29 });
```

Disabling individual scan rules

- If some scan rules are not required, we can disable those rules via the script.
- If your application contains many different preexisting violations of a specific rule, then consider disabling the rule.
- One can use AxeBuilder.disableRules() to temporarily disable individual rules until you're able to fix the issues.
- You can find the rule IDs to pass to disableRules() in the id property of the violations you want to suppress. A complete list of axe's rules can be found in axe-core's documentation.

Link: https://github.com/dequelabs/axe-core/blob/master/doc/rule-descriptions.md

```
1  import { test, expect } from '@playwright/test';
2  import AxeBuilder from '@axe-core/playwright'; // 1
3
4  test('Disable Rule', { tag: '@access8'}, async ({ page },testInfo) => {
5    await page.goto("https://app.box.com");
6    // await page.locator("id=login-email").fill('qacult.demo@gmail.com');
7    await page.getByLabel('Email Address', { exact: true }).fill('qacult.demo@gmail.com');
8
9    await page.locator("id=login-submit").click();
10
11   await page.locator("id=password-login").fill('testing123');
12   await page.getByRole('button', { name: 'Log In' }).click();
13
14   await page.locator("button[data-testid='new-item-menu-button']").waitFor();
15
16   const accessibilityScanResults = await new AxeBuilder({ page })
17       .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
18       .disableRules(['aria-required-children'])
19       .analyze();  // 4
20
21     expect(accessibilityScanResults.violations).toEqual([]); // 5
22   // await page.close();
23
24 });
```

Exporting scan results as a test attachment

- To include *all* of the scan results as part of your test results for debugging purposes, you can add the scan results as a test attachment with testInfo.attach().
- This will embed the scan results in the report.
- Reporters can then embed or link the full results as part of your test output.

```
1  import { test, expect } from '@playwright/test';
2  import AxeBuilder from '@axe-core/playwright'; // 1
3
4
5  test.describe('bbc page', () => { // 2
6    test('accessibility issues',{tag:"@access1"}, async ({ page },testInfo) => {
7      await page.goto('https://www.gov.uk/'); // 3
8
9      const accessibilityScanResults = await new AxeBuilder({ page })
10       .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
11       .analyze();  // 4
12
13       await testInfo.attach('accessibility-scan-results', {
14         body: JSON.stringify(accessibilityScanResults, null, 2),
15         contentType: 'application/json'
16       });
17
18     expect(accessibilityScanResults.violations).toEqual([]); // 5
19   });
20 });
```

Generate HTML reports

- To generate an HTML report using the above test results, you need to use another library called axe-html-reporter.
- Install **axe-html-reporter** using the following command:
- ➔ *npm install -D @axe-core/playwright axe-html-reporter*
- Modify the script to generate the HTML report:
  - *Import createHTMLReport from* axe-html-reporter
  - Save the html file in build/reports directory of your project.
- This will create an HTML report from Axe-core library AxeResults object listing violations, passes, incomplete and incompatible results.

```
const reportHTML = createHtmlReport({
results: accessibilityScanResults,
options: {
  projectKey: "YashHomepage"
},
});
const filename:string = `accessibility-report-${timestamp}.html`;
  let newPath = __dirname.replace('tests', ''); // Normalize path for
  newPath = path.join(newPath, 'build/reports'); // Ensure reports di
  const filepath: string = path.join(newPath, filename);

  fs.writeFileSync(filepath, reportHTML);
```

YASH Technologies SM

More than what you think.

Thank you!

© 2024 YASH Technologies | www.yash.com | Confidential